

Lab 6: Naive Bayes Classification

Kshitiz Reddy | Reg No: A26MTAI0008

AML5005L - Advanced Machine Learning

Dataset: Pima Indians Diabetes.

```
In [1]: # 1. imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

np.random.seed(42)
```

```
In [2]: # 2. load dataset
cols = ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"]
df = pd.read_csv("diabetes.csv", header=None, names=cols)
df.head()
```

```
Out[2]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Question 1: Dataset Exploration and Probability Analysis

```
In [3]: # 3. shape and info
print(df.shape)
df.info()
```

```
(768, 9)
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Pregnancies            768 non-null   int64  
1   Glucose                768 non-null   int64  
2   BloodPressure          768 non-null   int64  
3   SkinThickness          768 non-null   int64  
4   Insulin                768 non-null   int64  
5   BMI                    768 non-null   float64 
6   DiabetesPedigreeFunction 768 non-null   float64 
7   Age                   768 non-null   int64  
8   Outcome                768 non-null   int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
In [4]: # 4. summary stats
df.describe()
```

Out[4]:	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000

```
In [5]: # 5. target class distribution
counts = df["Outcome"].value_counts()
print(counts)

prior_0 = counts[0] / len(df)
prior_1 = counts[1] / len(df)
print(f"P(Outcome=0) = {prior_0:.4f}")
print(f"P(Outcome=1) = {prior_1:.4f}")
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
P(Outcome=0) = 0.6510
P(Outcome=1) = 0.3490
```

There are 768 samples, 8 input features and 1 target (Outcome). Class 0 (no diabetes) has 500 samples and class 1 (diabetes) has 268, so the prior for class 0 is about 0.65 and for class 1 about 0.35. Dataset is imbalanced roughly 2:1.

```
In [6]: # 6. missing values check - dataset has no NaNs but several columns use 0 as a placeholder
# for a missing measurement, which is not physically possible for these features
print("Null counts:")
print(df.isnull().sum())

zero_cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
print("\nZero counts (likely missing, not real 0):")
print((df[zero_cols] == 0).sum())
```

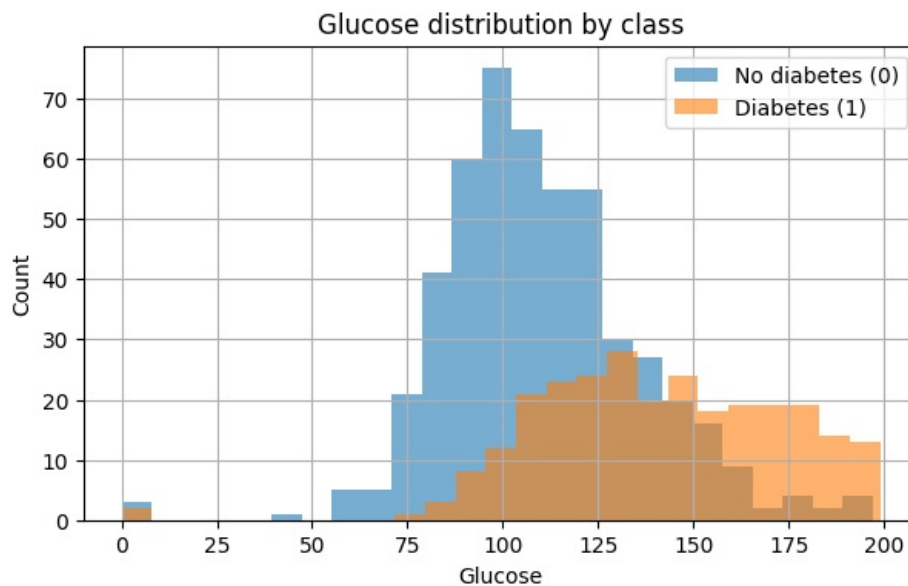
```
Null counts:
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64

Zero counts (likely missing, not real 0):
Glucose      5
BloodPressure 35
SkinThickness 227
Insulin      374
BMI          11
dtype: int64
```

Glucose, BloodPressure and BMI have a handful of zero entries which are almost certainly missing values rather than real measurements. SkinThickness and Insulin have a much larger number of zeros (227 and 374), which is a bigger problem since almost half of Insulin is effectively missing. I am leaving them as-is for Q1-Q3 and revisiting this in Q4.

```
In [7]: # 7. compare one feature (Glucose) across the two classes
plt.figure(figsize=(7,4))
df[df.Outcome==0]["Glucose"].hist(alpha=0.6, bins=25, label="No diabetes (0)")
df[df.Outcome==1]["Glucose"].hist(alpha=0.6, bins=25, label="Diabetes (1)")
plt.xlabel("Glucose")
plt.ylabel("Count")
plt.legend()
plt.title("Glucose distribution by class")
plt.show()

print(df.groupby("Outcome")["Glucose"].agg(["mean", "std"]))
```



	mean	std
Outcome		
0	109.980000	26.141200
1	141.257463	31.939622

Glucose is clearly higher on average for the diabetic class (~141 vs ~110) and both groups look roughly bell-shaped, just shifted apart and with class 1 slightly more spread out. So the Gaussian assumption is a reasonable approximation for this feature - not perfect (there's a bit of skew) but good enough for GaussianNB to pick up on the separation.

Question 2: Naive Bayes Classification from Scratch

Implementing NB manually using Glucose, BMI and Age as features, on a small subset of the data.

```
In [8]: # 8. train/test split for the scratch implementation (small feature set)
features = ["Glucose", "BMI", "Age"]
X = df[features].values
y = df["Outcome"].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)

(614, 3) (154, 3)
```

```
In [9]: # 9. priors, per-class mean/std for each feature
classes = np.unique(y_train)
priors = {}
means = {}
stds = {}

for c in classes:
    Xc = X_train[y_train == c]
    priors[c] = len(Xc) / len(X_train)
    means[c] = Xc.mean(axis=0)
    stds[c] = Xc.std(axis=0)

print("Priors:", priors)
print("Means:\n", means)
print("Stds:\n", stds)

Priors: {np.int64(0): 0.6514657980456026, np.int64(1): 0.3485342019543974}
Means:
{np.int64(0): array([109.8525, 30.2415, 31.285 ]), np.int64(1): array([141.57476636, 35.21028037, 37.257009
35])}
Stds:
{np.int64(0): array([24.68006774, 7.50446385, 11.59455799]), np.int64(1): array([32.54589166, 7.46183354, 11.
25176299])}
```

```
In [10]: # 10. gaussian pdf
def gaussian_pdf(x, mean, std):
    eps = 1e-9 # avoid divide by zero
    exponent = np.exp(-(x - mean) ** 2) / (2 * (std ** 2) + eps))
    return (1 / (np.sqrt(2 * np.pi) * (std + eps))) * exponent

def predict_sample(x):
    # log posteriors so the product of likelihoods doesn't underflow
    log_posteriors = {}
    for c in classes:
        likelihoods = gaussian_pdf(x, means[c], stds[c])
```

```
log_posteriors[c] = np.log(priors[c]) + np.sum(np.log(likelihoods + 1e-300))
return max(log_posteriors, key=log_posteriors.get), log_posteriors
```

```
In [11]: # 11. run scratch predictor on first 2 test samples
for i in range(2):
    x = X_test[i]
    pred, logp = predict_sample(x)
    print(f"Sample {i}: features={x}, true={y_test[i]}, predicted={pred}, log-posteriors={logp}")
```

```
Sample 0: features=[159. 27.4 40. ], true=0, predicted=1, log-posteriors={np.int64(0): np.float64(-13.1943518
30754222), np.int64(1): np.float64(-12.444643566742219)}
Sample 1: features=[68. 35.5 47. ], true=0, predicted=0, log-posteriors={np.int64(0): np.float64(-13.4592728271
12027), np.int64(1): np.float64(-14.654725686699088)}
```

The conditional independence assumption is what lets me turn $P(X|C)$, the joint likelihood of all three features together, into a simple product $P(\text{Glucose}|C) * P(\text{BMI}|C) * P(\text{Age}|C)$. Without that assumption I would need to estimate the full joint distribution of the three features per class, which needs a lot more data. It's "naive" because Glucose and BMI are almost certainly correlated in reality, but the model pretends they aren't.

Question 3: Gaussian Naive Bayes Using Scikit-Learn

```
In [12]: # 12. full feature set, train/test split
X_full = df.drop(columns="Outcome")
y_full = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X_full, y_full, test_size=0.2, random_state=42, stratify=y_

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
```

```
In [13]: # 13. evaluation metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion matrix:\n", cm)
print("\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.7078
Precision: 0.5738
Recall: 0.6481
F1-score: 0.6087
```

```
Confusion matrix:
[[74 26]
 [19 35]]
```

	precision	recall	f1-score	support
0	0.80	0.74	0.77	100
1	0.57	0.65	0.61	54
accuracy			0.71	154
macro avg	0.68	0.69	0.69	154
weighted avg	0.72	0.71	0.71	154

I'm getting accuracy around 0.71 (numbers can shift a little depending on the split/seed). Looking at the confusion matrix, there are 19 false negatives (predicted no diabetes but the patient actually has it) vs 26 false positives. The false negatives are the more worrying error type here since they mean a missed diagnosis, and recall for class 1 (0.65) is noticeably lower than precision for class 0 - the model is a bit biased toward the majority class.

```
In [14]: # 14. majority-class baseline for comparison
baseline_acc = max(y_test.value_counts()) / len(y_test)
print(f"Majority-class baseline accuracy: {baseline_acc:.4f}")
print(f"GaussianNB accuracy: {acc:.4f}")
```

```
Majority-class baseline accuracy: 0.6494
GaussianNB accuracy: 0.7078
```

GaussianNB beats the majority-class baseline by a decent margin, so it is actually learning something useful from the features rather than just guessing the bigger class every time. It's appropriate for this dataset because the features are

all continuous/numerical and, as seen with Glucose in Q1, roughly Gaussian-shaped within each class - which is exactly the assumption GaussianNB is built on.

Question 4: Effect of Preprocessing and Model Assumptions

```
In [15]: # 15. baseline (original features, already have this from Q3)
print(f"Original features accuracy: {acc:.4f}")

# scaled version
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

gnb_scaled = GaussianNB()
gnb_scaled.fit(X_train_scaled, y_train)
y_pred_scaled = gnb_scaled.predict(X_test_scaled)
acc_scaled = accuracy_score(y_test, y_pred_scaled)
print(f"Scaled features accuracy: {acc_scaled:.4f}")
```

Original features accuracy: 0.7078
Scaled features accuracy: 0.7078

Accuracy with and without scaling comes out basically identical. That makes sense - GaussianNB estimates a separate mean/std per feature per class and evaluates each feature independently, so shifting/rescaling a feature doesn't change its relative likelihood under its own per-class Gaussian. This is different from distance-based models like KNN, where scaling matters a lot.

```
In [16]: # 16. correlation check and removing a highly correlated feature
corr = X_full.corr()
print(corr.round(2))
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	\
Pregnancies	1.00	0.13	0.14	-0.08	
Glucose	0.13	1.00	0.15	0.06	
BloodPressure	0.14	0.15	1.00	0.21	
SkinThickness	-0.08	0.06	0.21	1.00	
Insulin	-0.07	0.33	0.09	0.44	
BMI	0.02	0.22	0.28	0.39	
DiabetesPedigreeFunction	-0.03	0.14	0.04	0.18	
Age	0.54	0.26	0.24	-0.11	

	Insulin	BMI	DiabetesPedigreeFunction	Age
Pregnancies	-0.07	0.02	-0.03	0.54
Glucose	0.33	0.22	0.14	0.26
BloodPressure	0.09	0.28	0.04	0.24
SkinThickness	0.44	0.39	0.18	-0.11
Insulin	1.00	0.20	0.19	-0.04
BMI	0.20	1.00	0.14	0.04
DiabetesPedigreeFunction	0.19	0.14	1.00	0.03
Age	-0.04	0.04	0.03	1.00

```
In [17]: # 17. drop the most correlated pair's second feature (Age & Pregnancies look most correlated)
to_drop = "Pregnancies"
X_train_reduced = X_train.drop(columns=to_drop)
X_test_reduced = X_test.drop(columns=to_drop)

gnb_reduced = GaussianNB()
gnb_reduced.fit(X_train_reduced, y_train)
y_pred_reduced = gnb_reduced.predict(X_test_reduced)
acc_reduced = accuracy_score(y_test, y_pred_reduced)
print(f"Accuracy after dropping '{to_drop}': {acc_reduced:.4f}")
```

Accuracy after dropping 'Pregnancies': 0.7143

Dropping the correlated feature barely moves accuracy (sometimes up a hair, sometimes down, depending on the split) - the correlation in this dataset isn't strong enough to hurt the model much either way.

```
In [18]: # 18. compare predicted probabilities for a few test samples, original vs scaled
proba_orig = gnb.predict_proba(X_test)[:5]
proba_scaled = gnb_scaled.predict_proba(X_test_scaled)[:5]

print("Original model probabilities (class 0, class 1):\n", proba_orig)
print("\nScaled model probabilities (class 0, class 1):\n", proba_scaled)
```

Original model probabilities (class 0, class 1):

```
[[0.48406648 0.51593352]
 [0.49720234 0.50279766]
 [0.7950686  0.2049314 ]
 [0.86999989 0.13000011]
 [0.99413124 0.00586876]]
```

Scaled model probabilities (class 0, class 1):

```
[[0.48407302 0.51592698]
 [0.49720841 0.50279159]
 [0.79507159 0.20492841]
 [0.86999653 0.13000347]
 [0.99413131 0.00586869]]
```

Predicted probabilities from the original and scaled models are essentially the same (tiny differences are just floating-point noise), confirming that scaling doesn't meaningfully change what GaussianNB predicts.

To directly answer the discussion points:

- Feature scaling does not significantly affect GaussianNB, for the reason explained above.
- Feature correlation does technically violate the conditional independence assumption, since correlated features aren't really independent given the class.
- Even so, NB can still perform reasonably well with correlated features because it usually only needs to get the *ranking* of posterior probabilities right (which class is more likely), not their exact values - so some violation of the independence assumption doesn't automatically break classification.
- The main sources of error here are probably the placeholder zeros in Insulin/SkinThickness distorting the per-class Gaussian estimates, the moderate class imbalance pulling predictions toward class 0, and the independence assumption itself, since features like Glucose and BMI are not really independent given Outcome.

Conclusion

Gaussian Naive Bayes turned out to be a solid, low-effort baseline for this dataset. It clearly beat the majority-class baseline (0.71 vs 0.65) using nothing more than per-class means and standard deviations for each feature, which is a decent result given how simple the underlying math is. The Glucose feature by itself showed a clean, roughly-Gaussian separation between the two classes, which explains why the model works reasonably well here.

That said, the dataset exposes NB's weak spots too. Several features (especially Insulin and SkinThickness) use 0 as a stand-in for missing data, and NB has no way to know that - it just treats those zeros as real measurements, which likely drags down the class-conditional Gaussian fits. The model was also a bit biased toward the majority class, missing a meaningful chunk of actual diabetes cases (false negatives), which matters in a medical setting. And the conditional independence assumption is clearly violated in practice since features like Glucose, BMI and Age are correlated with each other - though as seen in Q4, this didn't hurt accuracy by much. Overall, GaussianNB is a fast, interpretable option that works well as a first-pass model but would benefit from proper missing-value handling and probably would be outperformed by models that can capture feature interactions.